

Lab4 报告

1 实现目标与约束

Lab4 在 Lab1~Lab3 已完成的五级流水、真实访存、分支重定向与 Difftest 对齐框架之上，增量实现 RISC-V RV64 的 CSR 指令与关键机器态寄存器。根据测试要求，本阶段需要支持 CSRRW/CSRRS/CSRRC/CSRRWI/CSRRSI/CSRRCI 六条指令，并维护 mstatus、mtvec、mip、mie、mscratch、mcause、mtval、mepc、mcycle、mhartid、satp 等 CSR 的可见状态，同时将对对应字段接入 DifftestCSRState。实现过程延续此前实验的约束：不重构整体流水线，不引入新的复杂执行框架，不改变 Lab2/Lab3 已验证的访存与控制流主干，而是在现有模块边界内做最小侵入式扩展。

从语义上看，本次实现不仅要“能写 CSR”，还必须保证 CSR 指令写回通用寄存器时返回的是旧值，并覆盖 rs1=x0 与 zimm=0 的只读分支语义。同时，考虑到当前工程并未提供 CSR 专用旁路网络，Lab4 采用“CSR 写视为一种控制流改变事件”的策略：每次有效 CSR 写入触发一次流水线冲刷并从 pc+4 继续执行，以避免后续指令在旁路缺失时读取到过期 CSR 状态。

2 总体设计思路

整体设计继续使用 core.sv 中的 if_id_t/id_ex_t/ex_mem_t/mem_wb_t 作为唯一主流水通道，不额外拆出新模块。实现分成三条主线：第一条在译码级识别 CSR 指令类型与操作意图，把 csr_addr、操作类别、操作数来源与“是否真的写 CSR”信息编码进流水寄存器；第二条在执行路径上形成统一的 CSR 计算数据流，即先读旧值、再按操作生成候选新值、最后按 CSR 可写掩码收敛，并将旧值作为该条指令的 rd 写回数据；第三条在顶层提交语义附近维护 CSR 架构状态并完成 Difftest 接线，确保提交时刻看到的 CSR 与通用寄存器状态一致。

和 Lab3 相同，本次改动优先复用既有控制流重定向接口。fetch 的 redirect_valid/redirect_pc 不再只服务分支错预测，而是改为接收“分支错预测或 CSR 写入事件”的统一重定向请求；CSR 事件目标 PC 固定为当前 CSR 指令 pc+4。这种实现方式使 CSR 不需要引入专门的 hazard 单元，也避免在当前设计中额外扩展 CSR forwarding，符合实验的最小改动原则。

3 译码阶段扩展 (decode.sv)

译码模块在保持原有 ALU、访存、分支译码逻辑不变的前提下，补充了 CSR 专用控制信号输出：is_csr、csr_addr、csr_op、csr_use_imm、csr_we_intent。其中 csr_op 采用统一枚举编码 WRITE/SET/CLEAR/NONE，使后续执行级组合逻辑可以用同一套 case 处理六条 CSR 指令。

对 opcode=7'b1110011 的路径，译码先区分 funct3==000 与 funct3!=000。前者保留既有 ebreak/trap 识别，确保 Lab1~Lab3 的结束指令语义不被破坏；后者进入 CSR 指令译码。对于寄存器源操作数的 CSRRW/CSRRS/CSRRC，use_rs1 仍按原有风格拉高；对于立即数字段来源的 CSRRWI/CSRRSI/CSRRCI，通过 csr_use_imm 指示后续从 instr[19:15] 的 zimm 取值，不走寄存器读口。

`csr_we_intent` 在译码级就按规范约束收敛：`CSRRW/CSRRWI` 恒写，`CSRRS/CSRRRC` 仅在 `rs1!=x0` 时写，`CSRRSI/CSRRCI` 仅在 `zimm!=0` 时写。这样 EX 级只需叠加 `id_ex.valid` 与 `is_csr` 即可得到最终写使能，不会把“只读 CSR 指令”误当作实际 CSR 更新事件。

4 顶层执行与 CSR 数据通路 (`core.sv`)

本次实现将 CSR 数据计算放在 `core.sv` 的 EX 邻近路径，避免重写 `execute.sv` 端口并减少连锁改动。`id_ex_t` 增加了 CSR 控制字段，`ex_mem_t` 与 `mem_wb_t` 增加了 `is_csr/csr_we/csr_addr/csr_old_data/csr_new_data`，用于在流水推进过程中保持 CSR 指令完整语义信息。通用寄存器写回数据方面，保持原有 EX/MEM/WB 框架不变，仅在 EX 输出进入 EX/MEM 前插入一层轻量选择：若该条为 CSR 指令，则 `result` 使用 `csr_old_data`，从而保证 `rd` 总是写回修改前值。

CSR 读写逻辑通过两个集中函数实现。`csr_read_data()` 负责将 CSR 地址映射到当前架构状态值，并对 `mstatus/mtvec/mip` 等需要可见位约束的寄存器进行读取侧掩码处理；未实现寄存器统一返回 0。`csr_apply_mask()` 负责把操作结果收敛到合法写位：例如 `mip` 按 `MIP_MASK` 保留可写位，`mhartid` 始终只读，`mstatus/mtvec` 按头文件掩码更新，其余本实验要求支持且可写的寄存器按全宽更新。CSR 操作数来源统一为 `csr_src_data`：寄存器型取 `id_ex.op1`，立即数型取零扩展后的 `zimm`。

在更新时序上，CSR 架构寄存器放在 `core.sv` 时序块维护，`mcycle` 默认每拍自增，若同拍有对 `mcycle` 的 CSR 写入则由写入值覆盖自增结果，满足实验“写入优先于周期累加”的规则。`mhartid` 采用常量 0 连线，不参与写入路径。

5 CSR 写后冲刷与流水控制复用

CSR 不做 forwarding 是本次实现的显式策略，因此每次有效 CSR 写入都必须作为一次控制流边界处理。工程实现复用了 Lab3 已验证的 `redirect` 机制：当 EX 级识别到本条 CSR 指令确实发生写入 (`ex_csr_we`) 时，发出 `csr_redirect_valid`，并将 `csr_redirect_pc` 设为该条指令 `pc+4`。取指级统一接收 `redirect_valid = branch_mispredict | csr_redirect_valid`，并在 `csr_redirect_valid` 存在时优先选择 CSR 目标地址。

流水寄存器更新部分同样复用现有 flush 入口：原先仅 `branch_mispredict` 清空 IF/ID 与 ID/EX，现在扩展为“分支错预测或 CSR 重定向”都执行同样冲刷动作。这样既不会影响 Lab2 的 `mem_wait` 优先级与 load-use 插泡逻辑，也能保证 CSR 状态变化后，后续年轻指令不会在旧上下文中继续执行。

6 Difftest 对齐与 `coreid` 改接

`DifftestInstrCommit`、`DifftestArchIntRegState`、`DifftestTrapEvent`、`DifftestCSRState` 的 `coreid` 全部从常量 0 改为 `mhartid[7:0]`，与实验要求保持一致。`DifftestCSRState` 不再使用占位常量，而是接入真实 CSR 状态：`mstatus/mepc/mtval/mtvec/mcause/satp/mip/mie/mscratch` 均由当前架构寄存器驱动，`sstatus` 由 `mstatus & SSTATUS_MASK` 实时导出，S 态相关但本实验未实现的物理寄存器字段保持 0。

该接线方式延续了此前“提交点对齐”的整体风格：Difftest 看到的寄存器与 CSR 都来自同一套顶层状态源，避免出现“指令已提交但 CSR 仍是旧值”之类的观测错位。

7 验证与结果

```
Emu compiled at Mar 17 2026, 17:16:46
The image is ./ready-to-run/lab4/lab4-test.bin
Using simulated 256MB RAM
Using /home/jevonsshi/arch_2026/26-Arch/ready-to-run/riscv64-nemu-
interpreter-so for difftest
[src/device/io/mmio.c:19,add_mmio_map] Add mmio map 'clint' at [0x38000000,
0x3800ffff]
[src/device/io/mmio.c:19,add_mmio_map] Add mmio map 'uartlite' at
[0x40600000, 0x4060000c]
[src/device/io/mmio.c:19,add_mmio_map] Add mmio map 'uartlite1' at
[0x23333000, 0x2333300f]
The first instruction of core 0 has committed. Difftest enabled.
[WARNING] difftest store queue overflow
Core 0: HIT GOOD TRAP at pc = 0x8001fff8
total guest instructions = 32,767
instrCnt = 32,767, cycleCnt = 166,289, IPC = 0.197049
Seed=0 Guest cycle spent: 166,290 (this will be different from cycleCnt if
emu loads a snapshot)
Host time spent: 1,178ms
This emulator compiled with JTAG Remote Bitbang client. To enable, use
+jtag_rbb_enable=1.
Listening on port 23334
```